Advanced Task 2: React Foundation — Typed Components, Routing, and Tailwind System

Advanced Task 2 deliverable. Author: Ahmed. Date: 2026-11-14. Repository: https://github.com/pathlyworks/brandstore-ecommerce · Pull request: **#6** (`feature/react-foundation → main`). The plan it follows: `task 1 architecture.md`.

The React app lives in `web/` (an npm workspace), next to the legacy pages, which stay at the root until the cutover (task_1_architecture.md §9). It talks to the TechSphere API at `VITE_API_BASE`, read with `import.meta.env.VITE_API_BASE`, and falls back to `http://localhost:4010` (`web/src/config.ts`).

The brief asks for	Where
Vite + React + TypeScript app	`web/` (`vite.config.ts`, `tsconfig.json`: `strict`, `noImplicitAny`, `noUncheckedIndexedAccess`)
Tailwind with brand tokens for colours, spacing and typography	§1; `web/src/styles/theme.css` (generated from the legacy `css/base.css`), `web/scripts/check-tokens.mjs`
Core components with typed props (interfaces, generics where useful)	§2; `web/src/components/` — `Select<T>`, `Table<Row>`, `Tabs<T>` are generic
[v1.1] Storybook stories for every state, accessibility addon	§3; `web/src/components/*.stories.tsx`, `web/.storybook/`, `web/src/stories/stories.test.tsx`
React Router: layout route, nested routes, 404, matching the plan	§4; `web/src/app/routes.tsx`, checked against the plan by `web/src/app/routes.test.ts`
Lists with stable keys	§5; ESLint `react/jsx-key` and `react/no-array-index-key` as errors
Loading, empty and error states (conditional rendering)	§5; `components/StatusMessage.tsx`, every data route
Controlled forms	§5; catalog filters, quantity, sign-in
Data in a reusable hook, `useEffect` cleanup, no races or leaks	§6; `web/src/hooks/useApi.ts`, `useApi.test.tsx`
Tests and CI	§7; CI job "React app (web/)", Playwright projects `web-chromium` (and Firefox, WebKit in the cross-browser job)

1. Tailwind brand tokens

`npm run tokens:tailwind` reads the legacy design tokens (`css/base.css`) and writes the Tailwind v4 `@theme` (`web/src/styles/theme.css`, copy in `docs/migration/tailwind-theme.css`): 28 colours, 8 spacing steps (`3xs` to `2xl`), the font stack, 11 type sizes (`xs` … `h1`, fluid ones with `clamp()`), weights, line heights, letter spacing, radii, shadows and easings. Sizes outside Tailwind's namespaces (tap target, focus ring, z-index, durations) stay CSS variables and are used as `min-h-(--size-tap-target)`.

Only brand tokens can be used:

- The theme starts with `--color-*: initial;` and `--text-*: initial;`, so Tailwind's default palette and type sizes do not exist: `bg-blue-500` or `text-4xl` compile into nothing.
- `web/scripts/check-tokens.mjs` (part of `npm run lint -w web`) fails on arbitrary colour values (`bg-[#2563eb]`), default-palette class names, and raw hex, `rgb()`, `hsl()` or `oklch()` colours in `src/`.

2. Core components

All in `web/src/components/`, exported from `index.ts`. Props are interfaces; nothing is `any` (ESLint `no-explicit-any` and the type-checked typescript-eslint rules).

Component	Props (typed)	Generic	Accessibility
`Button`	`ButtonProps extends ComponentPropsWithRef<'button'>`: `variant`, `size`, `loading`, `loadingLabel`	—	`loading`: spinner, `aria-busy`, `aria-disabled`, stays focusable and ignores clicks
`Input`	`label`, `hint`, `error`, `loading` plus every native input prop	—	label always present; `aria-invalid`; hint and error linked with `aria-describedby`
`Select`	`options: SelectOption<T>[]`, `value: T`, `onChange(value: T)`	**`Select<T extends string>`**: a sort or a category cannot be set to a value the API rejects	native `<select>`, same label, hint and error wiring as `Input`
`Card`	`heading`, `href`, `media`, `footer`, `loading`, `disabled`, `tone`	—	with `href` the heading link is stretched over the card: one tab stop, named by the heading
`Badge`	`tone`, `loading`, `muted`	—	the meaning is in the text, never in the colour alone
`Modal`	`open`, `title`, `onClose`, `actions`, `error`, `busy`	—	`role="dialog"`, `aria-modal`, labelled by its title; focus trap; Escape; focus returns to the opener

Component	Props (typed)	Generic	Accessibility
`Tabs`	`tabs: TabItem<T>[]`, `value?: T`, `onChange(id: T)`	**`Tabs<T extends string>`**: the tab ids are a union, so `value` cannot name a missing tab	ARIA tabs pattern, roving tabindex, arrows, Home, End; disabled tabs skipped
`Table`	`columns: Column<Row>[]`, `rows: Row[]`, `getRowKey(row)`, `loading`, `error`, `emptyMessage`	**`Table<Row>`**: `cell` and `sortValue` receive a `Row`; one component for orders, cart and specs	`<caption>`, row headers, sort buttons with `aria-sort`
`Navbar`	`brand`, `links: NavItem[]`, `cartCount`, `account`, `loading`	—	`NavLink` (`aria-current="page"`), menu button with `aria-expanded`, Escape closes it
`Footer`	`groups: FooterGroup[]`, `note`	—	one labelled `<nav>` per group

The generics remove duplication the legacy code had: one `Table<Row>` replaces three hand-built tables (cart lines, orders, specifications), and one `Select<T>` serves the category, sort and cart-quantity selects with their own value types.

3. Storybook [v1.1]

`npm run storybook` (port 6006) or `npm run build-storybook -w web`. Every core component has a stories file with the six states. Hover and focus are shown with `data-force="hover"` / `"focus"`: `styles/index.css` redefines Tailwind's `hover:` and `focus-visible:` variants so they also match inside such an element, so a story shows the real styles without a pointer or a keyboard (nothing in the app sets `data-force`).

Component	Default	Hover	Focus	Disabled	Error	Loading	Extra stories
Button	✓	✓	✓	✓	✓ (danger action)	✓	Variants
Input	✓	✓	✓	✓	✓	✓	Controlled
Select	✓	✓	✓	✓	✓	✓	—
Card	✓	✓	✓	✓ (out of stock)	✓	✓ (skeleton)	—
Badge	✓	n/a	n/a	✓ (muted)	✓	✓	Tones
Modal	✓	✓	✓	✓ (busy)	✓	✓	Interactive
Tabs	✓	✓	✓	✓ (a disabled tab)	✓ (panel error)	✓ (panel loading)	—
Table	✓	✓	✓	✓ (no sortable columns)	✓ (message and retry)	✓ (skeleton rows)	Empty
Navbar	✓	✓	✓	n/a	n/a	✓ (cart count unknown)	SignedIn, MobileMenuOpen
Footer	✓	✓	✓	n/a	n/a	n/a	—

"n/a" states are not left blank: the stories file lists each in `parameters.states.notApplicable` with the reason (a badge is not interactive; the footer is static content and cannot fail or load; the navbar shows no fetched data).

Accessibility addon. `@storybook/addon-a11y` runs axe on every story in the Accessibility panel, with `a11y.test: 'error'` in `.storybook/preview.tsx`. The same checks run in CI: `src/stories/stories.test.tsx` renders every story with the Storybook preview (`composeStories`) and runs axe-core on it, and fails if

- a component is missing a stories file, or a state is neither a story nor a stated "n/a", or
- axe reports any violation (colour contrast is left to the addon and Lighthouse, because jsdom computes no styles).

4. Routing map

`web/src/app/routes.tsx`: one layout route (`RootLayout`: skip link, `Navbar`, `<Outlet />`, `Footer`; its `errorElement` is `RouteError`), the routes nested in it, a nested layout route for `/account` (`AccountLayout`: guard, account tabs, its own `<Outlet />`, index → `/account/orders`), and the `*` 404 route.

Path	Component	State in this task
`/`	`HomeRoute`	built: categories and best-rated products, two independent requests
`/products`	`ProductsRoute`	built: controlled filters in the URL, debounced search, grid, pages
`/products/:id`	`ProductRoute`	built: Tabs, specs `Table<Spec>`, controlled quantity, add to cart, 404 inside the page

Path	Component	State in this task
`/cart`	`CartRoute`	built: `Table<CartItem>`, quantity `Select`, remove, empty with a confirm `Modal`
`/checkout`	`CheckoutRoute`	placeholder (wave 4)
`/login`	`LoginRoute`	built: controlled form, validation, 401 message, `?next=` (same-site paths only)
`/register`	`RegisterRoute`	placeholder (wave 3)
`/account`	`AccountLayout`	built: nested layout, `RequireAuth`, sign out; index redirects to `/account/orders`
`/account/orders`	`OrdersRoute`	built: `Table<Order>`, five a page (the API pages by offset), 401 → sign in again
`/account/addresses`	`AddressesRoute`	placeholder (wave 3)
`/admin`	`AdminRoute`	placeholder (wave 4)
`/compare`	`CompareRoute`	placeholder (wave 5)
`*`	`NotFoundRoute`	built: inside the layout, names the path

The live map cannot drift from the plan. Each route carries `handle.component`. `routes.test.ts` reads the table in task_1_architecture.md §4 and fails unless the app has exactly those 13 paths with those component names. Placeholders say which wave migrates them; until then the legacy page serves that URL.

5. Keys, states and forms

- **Stable keys.** Every list is keyed by its data: products by `id`, categories by `id`, cart lines by `productId`, orders by `id`, table cells by column `id`, spec rows by label. `Table` takes `getRowKey`, so sorting keeps each row's identity (a test types into a row, sorts, and checks the text moved with the row). Index keys are an ESLint error (`react/no-array-index-key`); the only generated list, the skeleton, uses fixed slot ids.
- **Loading, empty and error.** `useApi` returns a status union (`idle`, `loading`, `success`, `error`), so a view cannot read `data` without checking the status first. `StatusMessage.tsx` gives one `LoadingState`, `EmptyState` and `ErrorState` (the message and next step per error code: 404, 429 with its wait, network, 5xx; "Try again" calls `retry`). While a new page loads the previous one stays visible (`aria-busy`, dimmed).
- **Controlled forms.** Catalog filters: the `Input` and both `Select<T>`s take their value from the URL search params and write back to them (search debounced by 350 ms, one history entry per pause). Product quantity: validated as you type (whole number, at least 1, no more than stock and the 10-per-line limit). Sign-in: values in state, errors shown on submit and updated as the field is corrected, the password cleared after a 401, and the request sent in the POST body only.

6. Data hook: `useApi`

```
const products = useApi(`products?${params}`, (signal) => fetchProducts(query, signal));
```

- The effect depends on `[key, attempt]` and nothing else: `key` describes the request, `attempt` is the retry counter. The loader is kept in a ref updated after each render, so an inline arrow does not re-run the effect and does not need to be a dependency (ESLint `react-hooks/exhaustive-deps` is an error).
- Each run creates an `AbortController`; the cleanup aborts it. A request for an old key is cancelled, and if its answer still arrives it is ignored (`signal.aborted`), so a slow page 1 can never overwrite page 2, and nothing sets state after unmount.
- `key: null` means "do not fetch yet" (no token).
- Errors become the app's `AppError` (`status`, `code`, `message`, `retryAfter`) from the API's `{error}` body.

The other effects clean up too: the debounce timer (`useDebouncedValue`), the document title, the cart's `storage` listener (`CartContext`), the session-expiry timer (`AuthContext`), and the key listeners, scroll lock and focus of `Modal` and `Navbar`.

A deliberate difference from the plan. task_1_architecture.md §4 lists React Router loaders for the data. This task asks for a reusable hook with `useEffect` cleanup, so the routes use `useApi`. The route objects are already the data router's (`createBrowserRouter`), so moving a route to a loader later changes that route only.

7. Tests and CI

Command	What it runs
`npm run test -w web`	Vitest, jsdom, React Testing Library, MSW (handlers over the captured TechSphere responses in `docs/migration/observed`): useApi (race, abort, unmount, retry), components, every route, the routing map against the plan, axe on every story
`npm run lint -w web`	ESLint (typescript-eslint type-checked, react-hooks, react keys) and the brand-token check
`npm run typecheck -w web`	`tsc` strict
`npm run build -w web`	`tsc` and `vite build`
`npm run build-storybook -w web`	the static Storybook
`npm run e2e`	Playwright: the legacy specs and `web/e2e/*.spec.ts` (routing, shopping, account) against the TechSphere API
`npm test`	the legacy Vitest suite, then the web suite

`npm run e2e` builds the React app with `VITE_API_BASE` and serves it with `vite preview` (port 4174, falls back to `index.html`, so client routes and the 404 work on reload). As before, the kit's server is started only when `VITE_API_BASE` is not set, so the grading machine needs Node only. `E2E_APPS=web` runs only the React specs.

CI (`.github/workflows/ci.yml`) adds the job **React app (web/)**: lint, typecheck, test, build, Storybook build (the Storybook is kept as a build artifact). The end-to-end job runs the web specs in Chromium and the cross-browser job in Firefox and WebKit.

8. Run it

```
npm install
npm run dev                          # the React app on http://localhost:5173 (API at VITE_API_BASE, default :4010)
VITE_API_BASE=http://127.0.0.1:4010 npm run dev
npm run storybook                    # http://localhost:6006
npm run dev:legacy                   # the legacy pages, as before
```

Test account: `student@techsphere.test` / `Passw0rd!`.